Availability 20 During a Busy Period

Operational constraints and method comparison

David Long | Supervisor report and presentation preparation
12 October 2026

Finding

I tested the two existing busy windows with a target of at least 20 available aircraft each day. The September to October window is achievable: an adapted priority-based search found a rule-compliant schedule with a busy-period minimum of 20 and a whole-forecast minimum of 18. The April to May window is infeasible under the current maintenance rules. The result depends on the dates being protected.

Busy period	Minimum 20	Evidence
27 April to 26 May 2033	Infeasible	Unavoidable D overlap caps availability at 18
14 September to 13 October 2033	Achieved	Saved feasible priority schedule reaches 20 every day

Requiring 20 in both windows simultaneously is therefore infeasible. The successful September to October schedule protects that window on its own.

Why the April to May window cannot reach 20

A D check lasts 126 days and may start only 70 days early. Every legal placement therefore shares 56 unavoidable maintenance days from its due date. Six aircraft have overlapping unavoidable intervals on 20 to 25 May 2033, leaving at most 24 minus 6 equals 18 available. Five D checks are unavoidable on several surrounding days, so even 20 is ruled out there. Lower availability before or after the window cannot remove these mandatory days.

What was allowed to change

As requested, I allowed lower availability outside the September to October window. Maintenance durations, legal start windows, due resets, check sequence and non-overlap remained mandatory. I also retained the loose scenario's 70-day displacement limit and its two dependency groups, each requiring at least two of four aircraft available every day. The original busy-period availability filters were replaced by the new target. No maintenance exemption was needed.

These are the existing dummy operational dates and groups. The surrounding period was measured over the 30 days immediately before and after the target; availability changes across the rest of the forecast were recorded as well.

The availability gained and the cost elsewhere

The original score maximises the lowest availability anywhere in the forecast. It can reject a useful busy-period improvement if availability falls elsewhere. In this experiment I first maximised the busy-period minimum up to 20, then minimised shortage below 20, then improved availability outside the period and smoothness. The target was allowed to be unmet during repair, but a reported success had to meet it on every day.

Measured priority result

Measure	As loaded	Earlier saved priority	Busy period priority
Busy-period minimum	15	18	20
Days below 20 in the period	25	30	0
Whole-forecast minimum	15	18	18
Minimum in 30 days before	18	19	18
Minimum in 30 days after	15	19	19
Forecast standard deviation	1.949767	1.413971	1.459788
Mean availability	21.713010	21.713010	21.713010

The adapted search first reached 20 after seven accepted priority moves, then continued improving the surrounding schedule. Its returned result used about 60 seconds and all checks passed maintenance replay, displacement and dependency validation. The largest displacement was 69 days.

What sacrificing nearby availability means here

The returned schedule gained 56 aircraft-days inside the busy period compared with the earlier saved priority export, and lost exactly that amount outside it. In the 30 days before the busy period, worst-day availability fell from 19 to 18; the 30 days after it stayed at 19. The whole-forecast minimum still remained 18. Thus this example did not require reducing the global minimum to 17 or breaking maintenance rules.

Compared with the as-loaded forecast, the busy period gained 65 aircraft-days and the rest of the forecast lost the same net amount. Other dates also improved, so net loss should not be confused with the sum of all individual daily reductions. Total maintenance downtime and whole-forecast mean availability did not change.

The earlier saved export is a historical maintenance-only reference, not the latest tuned result or a fair constrained competitor: it exceeds the current 70-day displacement limit on four checks. Its daily counts illustrate the trade-off; the new method comparison uses the same as-loaded start and retained constraints for both methods.

Whether annealing adds value

I also tested the existing basin-hopping candidate with the same adapted score, target-day priority, legal moves and retained constraints. Each run had a 60-second search budget, and the candidate used seeds 1, 2 and 3. Its warm-start and polish cap was reduced to 80 iterations to leave time for actual annealing escapes; the priority run retained its 400-iteration cap. This is an exploratory comparison under a time budget.

Method	Period min	Outside min	Std dev	Seconds
Priority	20	18	1.459788	60.3
Candidate seed 1	20	17	1.486601	60.2
Candidate seed 2	20	17	1.469236	60.2
Candidate seed 3	20	17	1.472725	60.3

All three candidate runs reached 20 and performed actual escape moves. Their median outside minimum was 17, and median full-forecast standard deviation was 1.472725. All reported plans passed maintenance and other retained operational checks. Every search reached its runtime budget, so these are best-so-far schedules rather than convergence claims.

Recommendation for this case

The adapted priority-based method is the practical choice for this tested window and budget. It already achieves 20 without reducing the whole-forecast minimum below 18; the candidate did not improve that result. The important change was giving the busy period priority in the objective. Annealing was not needed to find a feasible schedule here.

For another feasible window, annealing may help if improving the period requires several coordinated moves through temporarily worse schedules. It cannot overcome an unavoidable maintenance overlap. A definitive method ranking would need more date windows and the usual 300-second budget, with settings and acceptable surrounding availability specified in advance.

Relevant files

.scratch/busy-period-tradeoff/run_comparison.py

Reproducible isolated experiment. Published baseline and candidate files were not edited.

.scratch/busy-period-tradeoff/comparison-results.json

All reported measurements and experimental settings. Individual schedule JSON files are saved in the same folder.

.scratch/busy-period-tradeoff/window-check.json

Daily unavoidable D overlap and window feasibility bounds.

operational_constraints.json

Existing busy dates, displacement limits and dependency groups.